Can Deterministic Network Verification Reduce Undetected Faults in LLM-Generated Configurations?

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory study (study id c1-gnr-vv-2026-0001) to measure whether inserting a deterministic network verifier between an LLM intent→configuration generator and an emulated execution reduces the proportion of configurations that would execute with operational faults. Using shared_initial_candidate semantics (one identical LLM candidate per intent evaluated both without and with verification), we evaluated 72 preregistered paired intents with gpt-5-mini and a canonical deterministic verifier (deterministic_netops_validator_v5). Execution completed with 144 episodes (72 pairs) and 72 model calls. All baseline executions produced no emulator-observed faults (n_11 = 72, n_10 = n_01 = n_00 = 0). The paired difference in undetected-fault proportion is 0.0 (paired bootstrap 95% CI [0.0, 0.0], exact McNemar two-sided p = 1.0; bootstrap seed = 1729, resamples = 10000). Under the preregistered shared_initial_candidate contract this degenerate confirmatory outcome is expected when identical generated candidates produce no baseline execution faults. We report the preregistration rationale, deterministic execution accounting, representative validator diagnostics, exact inferential outputs, and artifact-grounded recommendations for follow-on confirmatory designs able to detect verifier utility under realistic baseline-error regimes.**

I. Introduction

Generative large language models (LLMs) are increasingly proposed to translate high-level operator intents into device-level network configurations. This intent→configuration automation can speed change pipelines but risks silently violating reachability, access-control, ordering, or workflow invariants and thereby causing outages if generated text is executed without additional checks. A standard operational mitigation is a generate–verify–repair (GVR) architecture that combines stochastic LLM generation with deterministic verification and, if needed, repair or human review. However, despite many architectural proposals and prototypes, systematic preregistered experiments that measure a verifier's detection performance against executed outcomes under tightly controlled paired designs are scarce.

This paper answers a narrowly scoped, preregistered research question: does inserting a deterministic network verifier between an LLM generator and emulator execution materially reduce the proportion of configurations that execute with operational faults when the identical generated candidate is evaluated both without and with verification? That estimand isolates verifier detection from improvements that could arise from re-sampling, prompt-engineering, or repair-enabled iterations. We executed study c1-gnr-vv-2026-0001 using gpt-5-mini and deterministic_netops_validator_v5 across 72 preregistered paired intents under shared_initial_candidate semantics. The run completed with no technical failures, produced a degenerate confirmatory outcome (no baseline emulator faults), and yields a paired difference of 0.0 (95% CI [0.0,0.0], exact McNemar p = 1.0). Rather than treating this as a null failure, we analyze why it occurred given the preregistered contract and archived artifacts, and we provide concrete, artifact-grounded guidance for designs that can detect verifier utility when baseline error prevalence is non-negligible.

II. Related Work

Prior work shows LLMs can perform intent→configuration translation in constrained vendor-dialect settings and emulator benchmarks [1]. Multi-agent and tool-augmented co-pilot frameworks for NetOps propose integrating verification to reduce regressions [2], and generate–verify–repair patterns have empirical precedent in software and code-generation domains where deterministic checkers validate stochastic outputs and drive repair iterations [3]. Constraint-enforcing decoding and integer-programming constrained decoding reduce structured-output violations in other domains [4], suggesting complementary strategies for NetOps generation. Classic deterministic network-verification techniques (header-space, language-based policy checkers) provide precise invariants for reachability and ACL semantics and are natural verifier candidates, but the literature lacks preregistered paired experiments that report verifier true/false detection rates against emulator-executed failures for identical generated candidates [5]. Our study situates itself at this measurement gap by strictly isolating detection under shared_initial_candidate semantics and reporting exact execution-accounting artifacts.

III. Methodology

Preregistration and estimand. We preregistered study c1-gnr-vv-2026-0001 and froze the design and analysis prior to observing outcomes. The primary estimand is the paired reduction in undetected operational-fault proportion (paired_success_rate_difference_guarded_minus_baseline).
The confirmatory hypothesis H1 targeted an absolute paired reduction of 0.20 with two-sided alpha = 0.05 and power >= 0.80; a sample-size heuristic returned ≈63 pairs and we preregistered N = 72 pairs (24 per topology-difficulty stratum) to allow margin for deterministic exclusions.

Shared_initial_candidate semantics and experimental contract. The execution_contract enforced shared_initial_candidate semantics: for each intent exactly one initial LLM generation was produced and that identical candidate was evaluated in two conditions. Baseline: execute the shared candidate in an isolated emulator and record emulator fault/no-fault. Guarded: run the canonical deterministic verifier (deterministic_netops_validator_v5) on the same candidate; if the verifier flags violations the guarded outcome is recorded as prevented (no execution); if it does not flag, execute the same candidate in the emulator and record the emulation outcome. This paired structure ensures any observed paired difference arises solely from verifier detection preventing an otherwise-executed fault.

Model, adapter, and verifier configuration. The declared model in execution/model_configuration.json is gpt-5-mini (provider = openai_responses, max_output_tokens = 2000, maximum_attempts_per_call = 1). The execution record explicitly sets temperature = null; we treat this as provider-default runtime sampling semantics and therefore as a residual, archived sampling-parameter uncertainty. The adapter family used was hosted_netops_gvr_v1. The canonical verifier was deterministic_netops_validator_v5 (validator_version = "5.0") configured to run the full_intent_constraint_validation rule set. The verifier emits structured validator_traces per episode that include parsed_command_count, required_command_count, normalized_configuration, validation_before/after final_state snapshots, violation_codes, and a difficulty.level tag used for stratification and diagnostics. Repair calls were permitted by the preregistration (maximum_repair_calls_per_task = 1) but none were applied during the confirmatory episodes (repair_applied = false in all archived traces).

Task-bank construction and contamination controls. Tasks were deterministically generated by deterministic_netops_task_generator_v5 (generator_version = "5.0"). Each task manifest encodes: a natural-language intent, an ordered machine-structured required_sequence, allowed_touched_fields, initial and expected final states, and a difficulty label (medium/hard/very_hard). The preregistered contamination procedure performed deterministic exact-match checks against public corpora; analysis/contamination_summary.csv reports zero exact-match exclusions for confirmatory pairs. Confirmatory stratification (24 pairs per stratum) was preserved as preregistered.

Execution protocol, retries, and deterministic accounting. The missingness_plan allowed up to two additional retries (3 attempts total) for transient hosted-API errors on generation calls and required full logging of retries and provider events. The run started at 2026-08-31T06:56:23.065707Z and completed at 2026-08-31T07:06:28.665518Z (≈10.1 minutes wall-clock). The execution manifest records planned_episode_count = 144, completed_episode_count = 144, failed_episode_count = 0, and model_calls_used = 72 (one shared generation per pair). Provider-call audit (deterministic_reconciliation.json) shows hosted_model_call_event_count = 72, completed_hosted_model_call_event_count = 72, cache_miss_count = 72, and stage_counts = {shared_initial_generation: 72}; no retries were consumed for confirmatory pairs.

Scoring, normalization, and binary outcome construction. For each episode the verifier produced a score and validator_trace. The primary analysis constructs binary indicators per pair as preregistered: baseline_undetected = 1 iff emulator execution of the shared candidate produced an operational fault; guarded_undetected = 1 iff the verifier did not flag a violation AND the emulator execution produced an operational fault. Verifier verdicts use normalized_configuration (deterministic normalization of whitespace and command ordering) and compare parsed_command_count against required_command_count derived from the task manifest; violation_codes enumerate rule failures (e.g., ordering_violation, protected_interface_touched). Validator traces for representative episodes show parsed_command_count == required_command_count and violation_count = 0 when the candidate satisfied the manifest constraints.

Inferential and bootstrap procedures. The preregistered primary inferential test for the paired binary estimand is an exact McNemar two-sided test when discordant pairs exist. We executed paired_binary_analysis_v1 to produce the 2×2 contingency table (guarded $\times$ baseline). Paired bootstrap-percentile 95% confidence intervals for the paired difference were computed with bootstrap_seed = 1729 and bootstrap_resamples = 10,000 (analysis/analysis_log.jsonl records these parameters). Pre-specified subgroup confirmatory comparisons by topology complexity and policy type were defined and registered with Holm correction, but they are non-informative in the absence of discordant pairs.

Sensitivity, deviations, and diagnostics. Pre-specified sensitivity analyses included: (1) complete_pair_primary_with_failure_as_zero_sensitivity which treats excluded pairs as failures, and (2) bootstrap diagnostics for verifier true/false detection rates. Because there were no exclusions (analysis/contamination_summary.csv) and no discordant outcomes (n_10 = n_01 = 0), these sensitivity analyses reproduce the degenerate numerical conclusion (paired difference = 0.0). Deterministic reconciliation checks passed: pair accounting and episode accounting are consistent and all deterministic consistency checks passed (deterministic_reconciliation.json all_deterministic_consistency_checks_passed = true). All scored episodes and validator_traces are archived to enable independent reexecution of the same analysis executor.

Artifact grounding. The manifest-level metrics cited above and the verifier configuration (validator_version = "5.0") are recorded in the archived evidence bundle. Representative per-episode traces under execution/scoring/ include normalized_configuration, validation_before/after final_state, parsed_command_count, required_command_count, difficulty.level, and repair_applied flags; these fields are the concrete primitives used by the deterministic scoring rules and the analysis scripts to form binary outcomes. The model con-

figuration file records declared_model_version = "gpt-5-mini" and temperature = null; we explicitly report temperature = null as an archived, provider-default runtime sampling semantics parameter that contributes to residual sampling uncertainty in interpretation.

IV. Results

Execution and manifest summary. The autonomous pipeline completed without technical failures. Key manifest figures (execution/ and analysis/ manifests): planned_episode_count = 144; completed_episode_count = 144; failed_episode_count = 0; model_calls_used = 72; artifact_count = 510. analysis/condition_summary.csv reports baseline successes = 72, baseline failures = 0 (success_rate = 1.0) and guarded successes = 72, guarded failures = 0 (success_rate = 1.0).

Paired contingency and exact inference. Aggregating across the 72 confirmatory pairs produced contingency counts n_11 = 72, n_10 = 0, n_01 = 0, n_00 = 0 (guarded $\times$ baseline). The paired estimate paired_success_rate_difference_guarded_minus_baseline = 0.0. Exact McNemar two-sided p = 1.0. The paired bootstrap percentile 95% CI (seed = 1729; 10,000 resamples) is [0.0, 0.0], reflecting the degenerate distribution produced by the absence of discordant pairs. analysis/paired_contingency_table.csv and analysis/condition_summary.csv contain these tabulations and are archived.

Representative validator diagnostics and per-episode exemplars. Representative per-episode scoring artifacts are archived (execution/scoring/task-000001-*.json ... task-000006-*.json). Representative summaries (extracted from archived JSONs): - pair-000001 (task-000001): difficulty.level = "medium", parsed_command_count = 4, required_command_count = 4, normalized_configuration matched the shared candidate, validator_version = "5.0", violation_count = 0. - pair-000002 (task-000002): difficulty.level = "hard", parsed_command_count = 2, required_command_count = 2, violation_count = 0. - pair-000003 (task-000003): difficulty.level = "hard", parsed_command_count = 6, required_command_count = 6, violation_count = 0. These traces show the verifier executed on each shared candidate and produced no violation flags across the sampled representative tasks. Execution/scoring/*.json files contain full validator_traces for reproducibility.

Contamination, missingness, and sensitivity diagnostics. analysis/contamination_summary.csv reports zero exact-match contaminations for confirmatory pairs. analysis/missingness_summary.csv records complete_pairs = 72 and zero failed or unscorable episodes. Pre-specified sensitivity analyses (treating excluded pairs as failures; bootstrap diagnostics) were executed and archived; because no pairs were excluded and no discordant outcomes occurred these sensitivity analyses reproduce the degenerate numeric conclusion (paired difference = 0.0). analysis/analysis_log.jsonl records bootstrap_seed = 1729 and bootstrap_resamples = 10000.

V. Discussion

Confirmatory interpretation and boundary conditions. The preregistered paired design with shared_initial_candidate semantics validly measures verifier detection on identical candidates; because the identical generated candidates produced zero emulator faults the verified paired outcome is a ceiling/null result: the verifier could not reduce executed faults that did not occur. The numeric outputs (n_11 = 72; paired difference = 0.0; McNemar p = 1.0; bootstrap CI [0.0,0.0]) follow directly from the preregistered contract and observed data. Reporting this degenerate outcome is important: it documents a reproducible case where the preregistered estimand and data-generating conditions leave no room for the verifier to demonstrate utility.

Artifact-grounded diagnostic factors. Archived artifacts allow concrete attribution of this ceiling outcome to measurable pipeline factors. First, the deterministic task generator (deterministic_netops_task_generator_v5) produced structured manifests with explicit required_sequence and allowed_touched_fields that constrain acceptable configs; these manifest constraints appear in execution/task_manifest.jsonl and increase the probability of a single correct generation. Second, the declared model (gpt-5-mini recorded in execution/model_configuration.json) produced candidates that consistently matched those structured constraints across the synthetic corpus (execution/responses/* files and per-episode shared_initial_candidate_sha256 entries). Third, the verifier's rule set (full_intent_constraint_validation in deterministic_netops_validator_v5) enforces the same required_sequence and field constraints encoded in the manifests; per-episode validator_traces under execution/scoring/ show parsed_command_count = required_command_count and violation_count = 0 for representative tasks. Together these aligned design choices—constrained intents, congruent verifier rules, and a model that produced matching sequences—explain the absence of baseline emulator faults.

Statistical and design implications. From a statistical perspective, a paired confirmatory test that targets a fixed absolute reduction (here preregistered 0.20) requires a nonzero baseline prevalence of executed faults to have power to detect a reduction; when baseline prevalence is zero the paired difference must be zero regardless of verifier performance. This is not a paradoxical failure of the verifier or analysis but an expected algebraic consequence of the estimand under shared_initial_candidate semantics. Practically, therefore, confirmatory designs intended to measure verifier detection should ensure the task bank yields a measurable baseline error rate (e.g., by including ambiguous intents, adversarial cases, or known failure-mode seeds) or should permit condition-specific sampling or repair so that the verifier can alter the executed candidate distribution.

Concrete preregisterable alternatives to detect verifier utility. The archived pipeline supports several clear modifications—each compatible with preregistration—that would permit mea-

suring different aspects of verifier utility without inventing new artifacts here: (1) independent-generation arms: allow separate initial generator calls per condition (baseline vs guarded) so that the verifier (or guarded workflow) can influence candidate distributions via repairs or sampling differences; (2) repair-enabled flows: permit a single deterministic repair call when the verifier flags a violation and then execute the repaired candidate to measure end-to-end improvement in execution outcomes; (3) adversarial/fault-injection task banks: seed tasks with intentionally ambiguous wording, ordering subtleties, or vendor-edge cases to raise baseline error prevalence and probe verifier detection under stress; (4) sampling-parameter sweeps and multi-model comparisons: record explicit non-null temperature and test multiple model versions to quantify model- and sampling-sensitivity; and (5) dedicated false-positive cost arms: measure operational blocking cost by comparing outcomes when verifier blocks flagged candidates versus when blocked candidates are repaired and re-executed. All of these are operationally meaningful, preregistrable changes and can be implemented using the same evidence-recording conventions shown in our run (execution/model_configuration.json, execution/scoring/, analysis/).

Threats to validity revisited with evidence. Internal validity: shared_initial_candidate semantics intentionally isolates verifier detection but prevents verifier-mediated resampling or repair effects; the deterministic provider-call audit (deterministic_reconciliation.json) confirms one shared generation per pair, making the isolation explicit. Construct validity: emulator-executed faults are a conservative surrogate for operational harm but do not capture traffic-dependent timing or vendor-hardware idiosyncrasies—limitations recorded in the manuscript. External validity: the run used a single declared model (gpt-5-mini) and a synthetic task bank; extrapolation to other models, richer real-world task distributions, or production hardware remains limited. Conclusion validity: with zero baseline faults the preregistered power analysis targeted to an absolute 0.20 reduction becomes moot; future confirmatory designs must align expected baseline prevalence with power targets.

Operational implications and measured tradeoffs. The observed ceiling does not imply that verifiers are unnecessary in practice. Instead, it shows that under constrained, high-clarity intents and a verifier rule set that encodes the same constraints, an LLM can produce device-level sequences that pass deterministic checks and emulator execution. In operational settings where intents are incomplete, telemetry is noisy, or vendor dialects vary, verifiers remain a principled guard. Measuring the verifier's net operational value requires explicit, preregistered trade-off quantification: estimate true-positive detection rates on a task bank with non-negligible baseline faults, and quantify verifier false-positive blocking costs (the proportion of flagged-but-safe candidates and operational delay or repair effort). The archived artifacts and deterministic accounting in this study provide a reproducible template to design such targeted, powered follow-on experiments.

VI. Limitations

- Confirmatory ceiling/null outcome: the baseline generator produced zero emulator faults on the preregistered task set, preventing direct measurement of verifier detection benefit.
- Shared_initial_candidate semantics: the design measures verifier effect on the same generated candidate only and does not assess independent per-condition generation, multi-sample generation, or repairs unless explicitly permitted by the contract.
- Single-model scope: experiments used one declared model (gpt-5-mini); results do not imply generality across models or versions.
- Synthetic/emulated tasks: task corpus and emulator executions differ from production devices and live traffic; external validity to production deployments is limited.
- Sampling-parameter uncertainty: execution/model_configuration.json shows temperature = null (sampling parameters unset); null indicates provider-default runtime sampling semantics and is a residual uncertainty recorded in the archived configuration.
- Residual contamination risk: deterministic provenance and exact-match checks were applied, but private training-data contamination of the hosted model cannot be fully ruled out and remains residual uncertainty.

VII. Conclusion

We executed a preregistered paired evaluation (c1-gnr-vv-2026-0001) under shared_initial_candidate semantics to test whether a canonical deterministic verifier reduces the proportion of LLM-generated configurations that would execute with operational faults. For the chosen model (gpt-5-mini), verifier (deterministic_netops_validator_v5), and synthetic/emulated task bank, baseline executions produced zero emulator faults and the verifier did not change outcomes (paired difference = 0.0; bootstrap 95% CI [0.0,0.0]; exact McNemar p = 1.0). This artifact-grounded null/ceiling outcome is internally consistent with the preregistered design: when identical generated candidates produce no executed faults a verifier cannot reduce executed faults under the shared_initial_candidate contract. Measuring verifier utility under realistic error regimes requires preregistered contracts that permit independent or multiple generator samples, repair-enabled flows, adversarial task sets, and multi-model comparisons. The archived artifacts (responses, task manifests, validator traces, execution manifest, and analysis logs) provide a reproducible baseline and a template for those follow-on confirmatory designs and power calculations.

Disclosure Statement

Autonomy and artifacts: This study was executed autonomously after an autonomy lock. Human role: a Research Director selected the topic family and approved the immutable initial master prompt prior to the autonomy lock; no human manually edited technical manuscript content after the lock. Models and tools:

initial generation used gpt-5-mini (declared_model_version recorded in execution/model_configuration.json) orchestrated via the hosted_netops_gvr_v1 adapter; deterministic_netops_validator_v5 (validator_version = "5.0") served as the canonical verifier; an isolated emulator executed candidate configurations. Preregistration: study c1-gnr-vv-2026-0001 was preregistered and frozen before observing outcomes. Immutable master prompt reference: provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Key archived artifacts include execution/raw_results.jsonl, execution/task_manifest.jsonl, execution/model_configuration.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, deterministic_reconciliation.json, and per-episode validator traces under execution/scoring/. The model configuration records temperature = null (provider-default sampling semantics archived in execution/model_configuration.json). Provider-call audit: hosted_model_call_event_count = 72. No human manual manuscript editing or content insertion occurred after the autonomy lock.

REFERENCES

[1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313

[2] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, Jian Yang, Ying Zhang, "Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework", 2025, 10.1145/3718958.3750537

[3] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899

[4] http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659